Backpressure, pipe & pipeline

10 min read Updated 24 Jul 2026 Topic 29 of 70

Bookmark Mark done

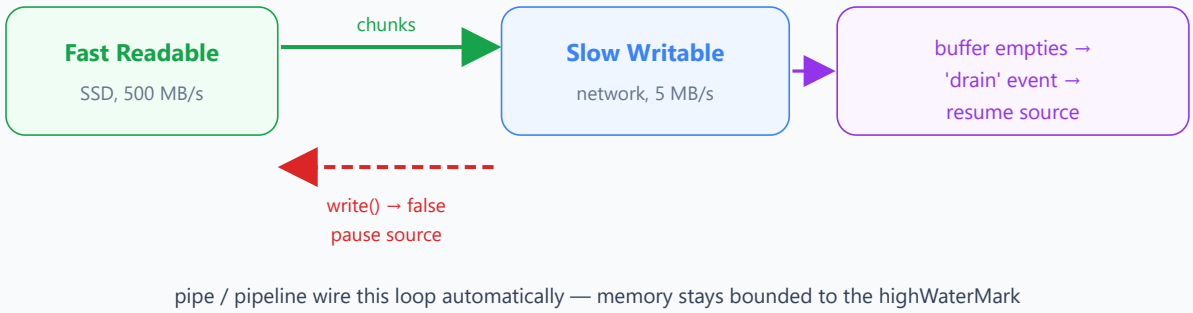
Why it matters

Backpressure is the single most important concept in the entire chapter, and the one most likely to be the deciding question in a senior Node interview. It is the reason streams stay memory-safe. Get it wrong and your service works flawlessly in testing, then silently accumulates gigabytes of buffered data under real load and gets OOM-killed — the most expensive category of production bug because it only shows up when the source is faster than the sink, which is exactly when traffic is high.

The connective tissue that manages backpressure for you is `pipe()` and its safer successor `pipeline()`. Knowing not just how to wire streams together but *why* `pipeline()` *exists and what* `pipe()` *gets wrong* is a hallmark of someone who has debugged real stream failures. This topic ties the whole chapter together: modes, buffers, highWaterMark, and events all exist to serve backpressure.

The concept, simply

Backpressure is what happens when a fast producer is connected to a slow consumer. Picture pouring water from a wide jug into a narrow funnel. If you pour faster than the funnel drains, water backs up and overflows. Backpressure is the funnel telling the jug "stop pouring until I catch up." In Node terms: a fast Readable (an SSD at 500 MB/s) feeding a slow Writable (a network socket at 5 MB/s) must be throttled, or the difference — 495 MB every second — piles up in memory.



Backpressure: the slow Writable signals write() false to pause the fast source, then emits drain when its buffer empties so the source resumes — memory stays bounded to the highWaterMark.

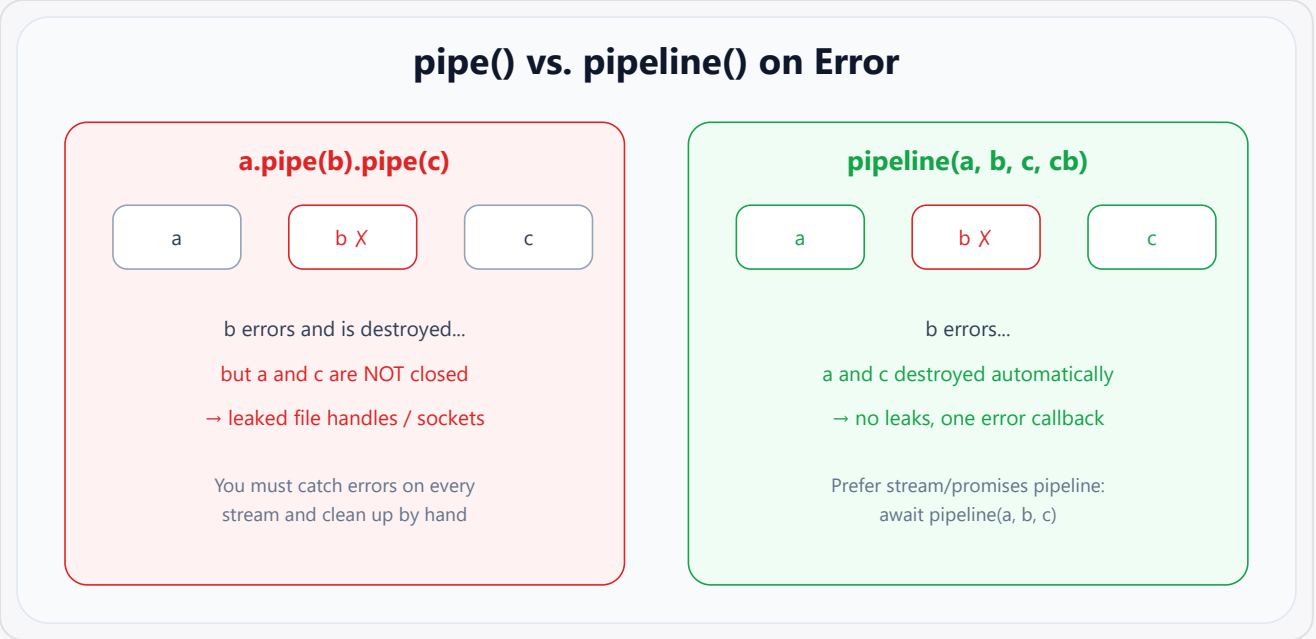
The signal is the Writable's `write()` return value. `false` means "my buffer is full, pause." The all-clear is the `'drain'` event: "my buffer emptied below the highWaterMark, resume." The whole protocol is: write until you get `false`, pause the source, wait for `'drain'`, resume. When you use `pipe()` or `pipeline()`, Node runs this loop for you automatically — you should almost never write it by hand.

How it works

The full signalling loop. When a Readable is piped to a Writable, each chunk it produces is passed to `dest.write(chunk)`. If that returns `false`, `pipe()` calls `source.pause()`, halting the flow of `'data'` events at the source (which in turn stops the source pulling from its underlying resource once its own buffer fills). The Writable keeps draining its buffer to the underlying resource in the background. When the buffered amount falls below the Writable's `highWaterMark`, it emits `'drain'`, and `pipe()` calls `source.resume()`. The net effect is that total buffered memory across the two streams is bounded to roughly the sum of their highWaterMarks — a few tens of kilobytes — no matter how large the data or how big the speed mismatch.

Why `pipe()` is not enough. `pipe()` handles backpressure correctly, but it handles *errors* badly. If a stream in the chain emits `'error'`, `pipe()` does **not** automatically destroy the other streams. Consider `a.pipe(b).pipe(c)`: if `b` errors, `a` and `c` are left open — their file descriptors or sockets leak, and under load those leaks exhaust the

manually destroy the others, which is verbose and easy to get wrong.



On error, pipe() leaves the other streams open (leaking descriptors); pipeline() destroys every stream in the chain and funnels the error to a single callback.

What `pipeline()` fixes. `stream.pipeline( ... streams, callback)` (and the promise form `pipeline` from `node:stream/promises`) wires up backpressure exactly like `pipe()`, but additionally: if any stream errors or closes early, it **destroys all the others** so nothing leaks; it forwards the error to a single callback (or rejects the promise); and it only reports success when the last stream has genuinely finished. It is the correct default for connecting three-plus streams, and even for two. Since Node 15+ the promise version lets you simply `await pipeline( ... )` inside a `try/catch`.

Client disconnects are backpressure too. When you stream a response and the client hangs up, the Writable (`res`) errors or closes. With `pipeline()`, the source file stream is destroyed automatically, so you stop reading a file nobody is listening to. With bare `pipe()`, that read continues, wasting I/O — another reason to prefer `pipeline()`.

Code examples

1. The wrong way — ignoring backpressure. This is the code that passes tests and OOMs in production.

```
import { createReadStream, createWriteStream } from 'node:fs';

const src = createReadStream('huge.bin');
const dst = createWriteStream('/dev/slow-disk/out.bin');

// BAD: pushes every chunk into dst regardless of its buffer state.
src.on('data', (chunk) => {
  dst.write(chunk);          // return value ignored — backpressure lost
});
src.on('end', () => dst.end());
// If dst is slower than src, dst's buffer grows without bound → memory bl
```

2. The right way — `pipeline()` from `node:stream/promises`. Backpressure, error propagation, and cleanup all handled.

JAVASCRIPT Copy

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';

try {
  await pipeline(
    createReadStream('huge.bin'),
    createGzip(),
    createWriteStream('huge.bin.gz')
  );
  console.log('Done — memory stayed flat, all streams cleaned up.');
} catch (err) {
  // ANY stream erroring lands here; every stream was already destroyed.
  console.error('Pipeline failed:', err.message);
}
```

3. Streaming an HTTP response safely, handling client disconnect.

JAVASCRIPT Copy

```
import { pipeline } from 'node:stream/promises';

createServer(async (req, res) => {
  try {
    await pipeline(createReadStream('./big.mp4'), res);
  } catch (err) {
    // Client disconnect (ERR_STREAM_PREMATURE_CLOSE) or read error.
    // pipeline already destroyed the file stream — nothing leaks.
    if (!res.headersSent) { res.statusCode = 500; res.end(); }
  }
}).listen(3000);
```

4. The manual drain loop — what `pipeline()` does for you. Write this once to understand it, then never again in real code.

JAVASCRIPT Copy

```
import { once } from 'node:events';
import { createReadStream, createWriteStream } from 'node:fs';

async function copyRespectingBackpressure(from, to) {
  const src = createReadStream(from);
  const dst = createWriteStream(to);
  for await (const chunk of src) {          // paused-mode: one chunk at a
    if (!dst.write(chunk)) {                // buffer full?
      await once(dst, 'drain');             // wait for it to empty
    }
  }
  dst.end();
  await once(dst, 'finish');
}
// pipeline() also handles errors and destroys both streams — this version
```

Common mistakes & gotchas

- **Ignoring the return value of `write()` in a manual loop.** The archetypal backpressure bug — memory grows unbounded, no error is thrown, and it only

manifests when the sink is slower than the source.

- **Using bare `pipe()` and assuming errors are handled.** They are not. A mid-chain error leaves the other streams open and leaking, and an unhandled `'error'` crashes the process.
- **Forgetting to clean up on error even with `pipe()`.** You must attach `'error'` to every stream and destroy the rest by hand — which is precisely why `pipeline()` exists.
- **Mixing `await pipeline()` with the callback form.** Import the promise version from `node:stream/promises` to `await`; the one from `node:stream` takes a callback and returns the last stream, not a promise.
- **Not handling client disconnects** when streaming responses — without `pipeline()`, the source keeps reading a file no one will receive.
- **Cranking the `highWaterMark` way up to "go faster."** It just raises the memory ceiling and delays backpressure; it does not increase real throughput past the sink's actual rate.

Best practices

- **Default to `pipeline()`** — specifically the promise version from `node:stream/promises` with `await` inside `try/catch` — for essentially all stream wiring, even two streams.
- **Never ignore the `write()` return value** in any hand-written loop; pause on `false`, resume on `'drain'`.
- **Let backpressure do its job** — don't defeat it by buffering chunks in an array or by raising the highWaterMark to hide a slow sink.
- **Handle errors in one place.** With `pipeline()` that is the catch/callback; avoid scattering `'error'` handlers across a bare `pipe()` chain.
- **For streaming responses, always account for client disconnect** — `pipeline()` tears down the source automatically so you stop doing useless I/O.
- **Reach for `stream/promises` helpers** (`pipeline`, `finished`) rather than manually listening for `'end'`/`'finish'`/`'close'`.

Q: What is backpressure and what mechanism implements it?
Backpressure is a slow consumer throttling a fast producer so buffered data stays bounded. The mechanism: `writable.write()` returns `false` when the internal buffer reaches the highWaterMark; the producer pauses and waits for the `'drain'` event before resuming. `pipe()` / `pipeline()` run this loop automatically.

Q: Why prefer `pipeline()` over `pipe()`? (The signature question.)
Both handle backpressure, but `pipe()` does not propagate errors or clean up: if one stream errors, the others stay open and leak descriptors, and an unhandled `'error'` crashes the process. `pipeline()` destroys every stream in the chain on any error or early close and reports the error to a single callback or rejected promise. It is safer and less code.

Q: In `a.pipe(b).pipe(c)`, `b` throws an error. What happens to `a` and `c`? (Trap.)
They are *not* automatically closed. `a` keeps reading and `c` stays open — leaking their underlying resources — and if `b`'s `'error'` has no listener the process crashes. This is the core reason to use `pipeline(a, b, c)`, which destroys all three.

Q: You ignore the boolean from `write()` and just keep writing. What breaks and when?
Nothing throws; the Writable's buffer grows past its highWaterMark without limit. Memory climbs and the process is eventually OOM-killed — but only when the sink is slower than the source, i.e., under real load, not in tests. It is a latent memory leak, not an immediate error.

Generally no. It raises how much data can buffer before backpressure kicks in — a bigger memory ceiling, not a faster sink. Real throughput is capped by the slowest stage. A larger mark can slightly reduce syscall overhead in some cases, but it mainly trades memory for delayed backpressure.

Q: How do you correctly handle a client disconnecting mid-stream while you serve a large file?

Use `await pipeline(fileStream, res)` in a `try/catch`. On disconnect the response Writable closes/errors, `pipeline` rejects (often `ERR_STREAM_PREMATURE_CLOSE`) and automatically destroys the file Readable, so you immediately stop reading a file no one will receive. Bare `pipe()` would keep reading.

Cheat-card

- **Backpressure** = slow sink throttles fast source so memory stays bounded.
- **Signal**: `write()` → `false` (buffer ≥ highWaterMark) → pause; `'drain'` → resume.
- `pipe()`: handles backpressure, *not* errors/cleanup — a mid-chain error leaks the other streams and can crash the process.
- `pipeline()`: backpressure *plus* destroys all streams on error and reports to one place. The correct default.
- **Use** `import { pipeline } from 'node:stream/promises'` → `await pipeline(a, b, c)` in `try/catch`.
- **Don't** ignore `write()`'s return, buffer chunks to defeat backpressure, or raise highWaterMark to "go faster."

QUICK CHECK — no marks, just for you.

1. What is backpressure and what signal implements it?

It is a slow consumer throttling a fast producer; writable.write() returns false when the buffer hits the highWaterMark, and the producer resumes on the "drain" event

- It is the OS refusing new TCP connections, signalled by an EADDRINUSE error
- It is the garbage collector pausing the stream, signalled by a "gc" event
- It is the readable emitting "error" whenever the buffer overflows

Check answer

2. Why prefer pipeline() over pipe()?

- pipeline() is faster because it skips the internal buffer entirely
- Both handle backpressure, but pipe() does not propagate errors or clean up; pipeline() destroys every stream on any error and reports to a single callback or rejected promise
- pipe() cannot connect more than two streams, but pipeline() can
- pipeline() disables backpressure so data moves without throttling

Check answer

3. In a.pipe(b).pipe(c), stream b emits an error. What happens to a and c?

- Both a and c are automatically destroyed by pipe()
- Only c is closed; a continues but is harmless
- They are not automatically closed; a keeps reading and c stays open, leaking their resources, and an unhandled error on b can crash the process
- Node restarts the pipeline from a automatically

Previous
Transform & Duplex Streams

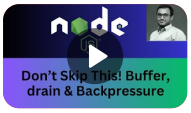
Next
The http module & createServer

LEARN MORE

ARTICLES & DOCS

- Backpressuring in Streams | Node.js Learn
  nodejs.org
- Stream | Node.js Documentation
  nodejs.org

VIDEOS



Node.js Streams EXPLAINED: Backpressure, Buffer Overflow & drain Event
Leela Web Dev

Hand-picked references for this lesson.